



CL-2001 Data Structures Lab # 7

Objectives:

- Stack
- Stack application

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. There must be a block of comments at start of every question's code by students; the block should contain brief description about functionality of code.
2. Comment on every function about its functionality.
3. Use understandable name of variables.
4. Proper indentation of code is essential.
5. Write a C++ statement(s) for each of the following task one after the other, in the same order.
6. Make a Microsoft Word file and paste all of your C++ code with all possible screenshots of every **task output in MS word and submit .cpp file with word file.**
7. Make separate .cpp files for all tasks and use this format **22F-1234_Task1.cpp.**
8. First think about statement problems and then write/draw your logic on copy.
9. After copy pencil work, code the problem statement on MS Studio C++ compiler.
10. At the end when you done your tasks, attached C++ created files in MS word file and make your submission on Google classroom. (Make sure your submission is completed).
11. Please submit your file in this format **22F-1234_L1.**
12. Do not submit your assignment **after the deadline.**
- 13. Do not copy code from any source otherwise you will be penalized with negative marks.**



Problem: 1 | Array based Stack

You are required to implement this Undo/Redo operation using a stack data structure. Let's assume you are dealing with a simple text editor that supports the insertion of text, deletion, and alteration of existing text. Each action is represented by a command that the user can perform. The data structure you create should be capable of pushing each command into the stack and be able to undo or redo the actions using traditional Stack operations.

1. `push(command)`: This function pushes a command into the stack.
2. `pop()`: This function pops the last command from the stack, simulating the Undo operation.
3. `peek()`: This function returns the top command from the stack without removing it.
4. `isEmpty()`: This function checks if the stack is empty.
5. `isFull()`: This function checks if the stack is full.
6. `size()`: This function returns the current size of the stack.

Note: For simplicity, a command can be represented as a string, e.g., "insert 'text'", "delete 'text'", "alter 'text1' to 'text2'". The pop function simulates the Undo operation by returning the last command performed, but it's up to the user to handle this command and revert its action.

Problem: 2 | Linked list-based stack

Web browsers allow users to navigate back and forth through the history of viewed pages. This is usually implemented with the help of two stacks: one stack for storing the pages that can be reached by pressing the 'Back' button, and another stack for the pages that can be reached by pressing the 'Forward' button. Your task is to implement this feature using two stacks, each implemented with a linked list.

Your implementation should support the following operations:

1. `visit(page)`: This function receives a string 'page', representing the URL of a new page to visit. Upon visiting a new page, all the pages in the 'Forward' stack are removed.
2. `back()`: This function simulates the pressing of the 'Back' button. It should return the previous page visited and move it from the 'Back' stack to the 'Forward' stack. If there's no previous page, it should return a specific message, like "No previous page".
3. `forward()`: This function simulates the pressing of the 'Forward' button. It should return the next page and move it from the 'Forward' stack to the 'Back' stack. If there's no next page, it should return a specific message, like "No next page".
4. `current()`: This function should return the current page without altering any data.

Problem: 3 | Stack applications

Stacks are used by compilers to help in the process of evaluating expressions and generating machine language code. In this and the next exercise, we investigate how compilers evaluate arithmetic expressions consisting only of constants, operators and parentheses.

Humans generally write expressions like $3 + 4$ and $7 / 9$ in which the operator (+ or / here) is written between its operands—this is called infix notation. Computers “prefer” postfix notation in which the operator is written to the right of its two operands. The preceding infix expressions would appear in postfix notation as $3\ 4\ +$ and $7\ 9\ /$, respectively.



To evaluate a complex infix expression, a compiler would first convert the expression to postfix notation and evaluate the postfix version of the expression. Each of these algorithms requires only a single left-to-right pass of the expression. Each algorithm uses a stack object in support of its operation, and in each algorithm the stack is used for a different purpose.

In this exercise, you'll write a C++ version of the infix-to-postfix conversion algorithm. In the next exercise, you'll write a C++ version of the postfix expression evaluation algorithm. Later in the chapter, you'll discover that code you write in this exercise can help you implement a complete working compiler

Write a program that converts an ordinary infix arithmetic expression (assume a valid expression is entered) with single-digit integers such as

$(6 + 2) * 5 - 8 / 4$

to a postfix expression. The postfix version of the preceding infix expression is

$6 2 + 5 * 8 4 / -$

The program should read the expression into string infix and use modified versions of the stack functions implemented in this chapter to help create the postfix expression in string postfix. The algorithm for creating a postfix expression is as follows:

1) Push a left parenthesis '(' onto the stack. 2) Append a right parenthesis ')' to the end of infix. 3) While the stack is not empty, read infix from left to right and do the following:

If the current character in infix is a digit, copy it to the next element of postfix. If the current character in infix is a left parenthesis, push it onto the stack. If the current character in infix is an operator, Pop operators (if there are any) at the top of the stack while they have equal or higher precedence than the current operator, and insert the popped operators in postfix.

Push the current character in infix onto the stack. If the current character in infix is a right parenthesis Pop operators from the top of the stack and insert them in postfix until a left parenthesis is at the top of the stack. Pop (and discard) the left parenthesis from the stack.

The following arithmetic operations are allowed in an expression:

+ addition

- subtraction

* multiplication

/ division

^ exponentiation

% modulus

[**Note:** We assume left-to-right associativity for all operators for the purpose of this exercise.] The stack should be maintained with stack nodes, each containing a data member and a pointer to the next stack node.



Some of the functional capabilities you may want to provide are:

- a) function `convertToPostfix` that converts the infix expression to postfix notation
- b) function `isOperator` that determines whether character is an operator
- c) function `precedence` that determines whether the precedence of `operator1` is greater than or equal to the precedence of `operator2`, and, if so, returns true.
- d) function `push` that pushes a value onto the stack
- e) function `pop` that pops a value off the stack
- f) function `stackTop` that returns the top value of the stack without popping the stack
- g) function `isEmpty` that determines if the stack is empty
- h) function `printStack` that prints the stack

Driver Code:

```
#include <iostream>
#include <conio.h>
#include <string.h>
using namespace std;
struct node
{
    int data;
    node *next;
}*p = NULL, *top = NULL, *save = NULL, *ptr;
void push(int x)
{
    p = new node;
    p->data = x;
    p->next = NULL;
    if (top == NULL)
    {
    }
    else
    {
    }
}
char pop()
{
    if (top == NULL)
    {
    }
    else
    {
    }
}
```



```
}  
int main()  
{  
    char expr[30];  
    cout << "enter the balanced expression\n";  
    cin >> expr;  
    for (int i = 0; i < strlen(expr); i++)  
    {  
  
    }  
    cout << "ans is " << pop() << endl;  
  
    system("pause");  
}
```